PE-JV-003

Aula 02 - APIs REST resilientes e Arquitetura

Aquecimento

Kahoot

Revisão da Aula 01: Java 21, Records, Value Objects, Pattern Matching e a base sobre Virtual Threads.
Aguarde o link do professor.

O Desafio de Engenharia

Vamos estruturar a integração da API de Checkout. O domínio de dados e as regras de negócio foram mapeados pelo time de Produtos. O foco é técnico. A API precisa aguentar 10 mil usuários simultâneos e depende de um Serviço de Pagamento externo (ex: Cielo, Stripe).

Três desafios:

1. Qual paradigma usar para aguentar essa carga?
2. Como protegemos nosso sistema se o serviço de pagamento externo cair ou ficar lento?
3. O que acontece se o usuário reenviar a mesma requisição em caso de lentidão?

Documento de Projeto

No seu documento de projeto (System Design):

- Definir o paradigma da API (MVC vs Reativo).
- Desenhar as travas de segurança e resiliência.

Pesquisa Individual

Acesse os links compartilhados e foque nos seguintes pontos:

1. Qual a diferença entre Spring MVC e Spring WebFlux? Virtual Threads mudam algo na análise?
2. Busque sobre mecanismos de resiliência (ex: Resilience4j). Além do *Circuit Breaker* e *Retry*, o que fazem o *TimeLimiter*, *Bulkhead* e *RateLimiter*? Como funciona a *Idempotency-Key* num `POST` ?
3. Como testar a API para garantir que nosso sistema não cai quando a API externa cai? (Busque: WireMock ou Fault Injection).

Cenários de Risco

Durante a sua leitura, adote a mentalidade da Engenharia do Caos.

- Formule 3 perguntas de risco relacionadas ao cenário de pagamento e anote.
- Exemplo: "E se der timeout no retry, cobramos o cliente duas vezes?"
- Exemplo: "E se o banco de dados cair na hora de registrar o estado de falha?"

Debate

É hora de debater os achados da pesquisa:

- **MVC vs Reativo:** O que muda com as Virtual Threads?
- **Edge Cases:** Vamos testar a arquitetura contra as 3 perguntas de risco que vocês criaram.
- **Além do Circuit Breaker:** Por que e quando utilizar *TimeLimiter*, *RateLimiter* ou *Bulkhead*? Qual recurso faz sentido no nosso cenário?

Projeto Individual (System Design)

Acesse o Template de System Design da Aula 02 (disponível no LMS). Não foque em escrever contratos extensos (Swagger/OpenAPI).

Entrega e Versionamento:

1. Crie um repositório público no GitHub com o padrão:
   `ada-pe-jv-003-system-design-[seu-nome]`
2. Envie o link do repositório na Planilha compartilhada pelo professor no chat.

Passo a passo dos commits:

- **Commit 1:** Crie e commite o arquivo com as suas 3 perguntas de risco.
- **Commit 2:** Escreva e commite o System Design contendo:

Prática de Código (Opcional)

Disponibilizamos um artefato auxiliar no LMS contendo um desafio prático de código. O objetivo é criar uma API com Spring Boot e Resilience4j, forçar a queda da chamada externa e ver a abertura do Circuit Breaker e a atuação do Fallback.

Próxima Aula: Banco de Dados

Agora temos a API preparada para lidar com falhas externas. Onde e como vamos salvar os dados com segurança em altíssima concorrência?

Na próxima aula:

- ACID vs Consistência Eventual no pagamento.
- Como as estratégias do JPA impactam a escala? (Auto-increment vs UUID vs ULID).
- **SQL vs NoSQL:** Redis, Elasticsearch, grafos e o ecossistema NoSQL.
- **PostgreSQL:** Padrões de otimização, índices, concorrência e uso de *Locks*.
- Volumetria, expurgo de dados antigos e análise de performance (usando `EXPLAIN ANALYZE`).

Obrigada